

Linux Device Drivers (3. Baskı)

Detaylı Çalışma Kağıdı – Bölüm 2–6, 8, 10, 14

Ahmet Hasan Çelik
Gömülü Sistem Stajı – Kernel Modülü Geliştirme

4 Ağustos 2026

İçindekiler

1	Modülleri Derleme ve Çalıştırma	4
1.1	Hello World: Bir Modülün Anatomisi	4
1.2	Neden <code>printf</code> değil de <code>printk</code> ?	4
1.3	Uygulama ile Modül Arasındaki Fark: Olay Güdümlü Model	4
1.4	Kernel Stack’inin Küçüklüğü	4
1.5	<code>current</code> : Şu Anki Process	4
1.6	<code>Kbuild</code> : Kernel Modülü Neden Normal <code>gcc</code> ile Derlenmez	5
1.7	<code>insmod</code> , <code>modprobe</code> , <code>rmmod</code> , <code>lsmod</code> – Gerçekte Ne Yaparlar	5
1.8	Modül Parametreleri	6
1.9	Kernel Sembol Tablosu ve Modül Yığılması (Stacking)	6
1.10	Hata Yönetimi: <code>goto</code> Kalıbı ve Sökme Sırası	6
1.11	Kullanıcı Alanında mı, Kernel’de mi Yazmalı? (Doing It in User Space)	7
2	Karakter Sürücüler	9
2.1	<code>scull</code> ’un Tasarımı: Neden Birden Fazla “Kişilik”?	9
2.2	Major/Minor Numaralarının İç Temsili	9
2.3	Cihaz Numarası Ayırma: <code>register_chrdev_region</code> vs <code>alloc_chrdev_region</code>	9
2.4	Üç Temel Veri Yapısı: <code>file_operations</code> , <code>file</code> , <code>inode</code>	10
2.4.1	<code>struct file_operations</code>	10
2.4.2	<code>struct file</code> : <code>f_pos</code> , <code>f_flags</code> , <code>private_data</code>	10
2.5	<code>cdev</code> : Sürücüyü Kernel’e Tanıtmak	11
2.5.1	<code>container_of</code> : <code>inode</code> ’dan kendi <code>struct</code> ’na geri dönmek	11
2.6	<code>scull</code> ’un Bellek Yönetimi: Quantum ve Qset	12
2.7	<code>read</code> ve <code>write</code> : Tam Algoritmik Detay	12
2.7.1	Neden <code>buff</code> ’a doğrudan dokunulamaz? (3 gerekçe, kitaptan)	12
2.7.2	<code>ssize_t</code> dönüş değeri sözleşmesi (hem <code>read</code> hem <code>write</code> için)	13
2.8	<code>scull</code> ile Oynamak: <code>strace</code> Önerisi	14
3	Hata Ayıklama Teknikleri	15
3.1	<code>printk</code> : Log Seviyeleri	15
3.1.1	<code>printk</code> nasıl saklanır: dairesel tampon (circular buffer)	15
3.1.2	<code>printk_ratelimit</code> : log taşmasını önlemek	15
3.2	Koşullu Debug Mesajları: <code>PDEBUG</code> Kalıbı	15
3.3	<code>/proc</code> Dosya Sistemi ile Sorgulama	16

3.4	ioctl ile Hata Ayıklama	16
3.5	strace: Sistem Çağrılarını Dışarıdan İzlemek	16
3.6	Oops Mesajlarını Okumak	16
3.7	Sistem Kilitlenmeleri: Magic SysRq	17
3.8	Debugger'lar: gdb, kdb, kgdb, UML	17
4	Egzamanlılık ve Race Condition'lar	19
4.1	scull'daki Somut Race Condition	19
4.2	Genel Kural: Paylaşılan Kaynak = Yönetim Zorunluluğu	19
4.3	Semaphore ve Mutex	19
4.3.1	scull'da semafor kullanımı	20
4.4	Reader/Writer Semaphore (rwsem)	20
4.5	Completion: "İş Bitti" Bildirimi	20
4.6	Spinlock: Uyuyamayan Kod İçin Kilit	21
4.7	Kilitleme Tuzakları	21
4.8	Kilitlemeye Alternatifler	22
5	Gelişmiş Karakter Sürücü İşlemleri	23
5.1	ioctl: Genel Bakış	23
5.2	Komut Numarası Kodlamaşı: Neden Bu Kadar Karmaşık?	23
5.3	ioctl Argümanının Güvenli Kullanımı	24
5.4	Capabilities: root Yerine İnce Taneli Yetki	24
5.5	Blocking I/O: Process'i Uyutmak	24
5.5.1	Üç temel kural (kitaptan, aynen)	24
5.5.2	wait_event ailesi	25
5.5.3	sleepy örneği: paylaşılan bayrak race'i	25
5.5.4	O_NONBLOCK ve -EAGAIN	25
5.5.5	sculpipe: Gerçek Bir Bloklayan Okuma Örnek	25
5.5.6	Elle Uyuma Mekanığı (wait_event'in altında ne oluyor)	26
5.6	poll ve select	26
5.7	Asenkron Bildirim (fsync / SIGIO)	27
5.8	Cihazı Aramaya Kapatmak: nonseekable_open	27
5.9	Cihaz Dosyası Üzerinde Erişim Kontrolü	27
6	Bellek Ayırma	29
6.1	kmalloc: Kernel'in malloc'u, Ama Farklı Kurallarla	29
6.2	Slab Cache: Aynı Boyuttaki Nesneleri Sık Sık Ayırıp Bırakanlar İçin	29
6.3	Sayfa Bazlı Ayırma ve vmalloc: Farklı İhtiyaçlar İçin Farklı Araçlar	29
6.4	CPU-Başına Değişkenler: Kilitsiz Hız	29
6.5	Büyük Tamponlar: Neden Kaçınmalı	29
7	Kesme (Interrupt) Yönetimi	30
7.1	Kesme Neden Var: Polling'e Alternatif	30
7.2	Handler Kuralları: Neden Bu Kadar Kısıtlı	30
7.3	Kesme ile Bloklayan I/O'nun Buluştuğu Yer	30
7.4	Paylaşılan Kesmeler	30
8	The Linux Device Model (Aygıt Modeli)	31
8.1	kobject: Her Şeyin Altında Yatan Ortak İskelet	31
8.2	sysfs: Neden Her kobject Bir Dizindir	31
8.3	Hotplug Olayı: udev'in /dev Düğümünü Nasıl Oluşturduğu	31
8.4	Bus → Device → Driver: PCI/USB'de Gördüğünün Genelleşmiş Hali	31
8.5	class: class_create'in geçmişi ve senin kodunla farkı	31

8.6 Firmware Yükleme (Kısa Not)	31
Sonuç: Kitaptan ahmet.c'ye Dönen Bir Yol Haritası	33

Bölüm 1 Modülleri Derleme ve Çalıştırma

Hello World: Bir Modülün Anatomisi

Kitap ilk örneğini şu şekilde verir:

```
#include <linux/init.h>
#include <linux/module.h>
MODULE_LICENSE("Dual BSD/GPL");

static int hello_init(void)
{
    printk(KERN_ALERT "Hello, world\n");
    return 0;
}

static void hello_exit(void)
{
    printk(KERN_ALERT "Goodbye, cruel world\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

Kritik nokta: fonksiyonların adı `hello_init`/`hello_exit` olması bir zorunluluk değil – `module_init()` ve `module_exit()` birer *makro*dur ve kernel'e "yüklenince/kaldırılınca hangi fonksiyonu çağıracağım" söyler. Sen bu fonksiyonlara istediğin ismi verebilirsin (senin kodunda `ahmet_init`/`ahmet_exit`).

Neden printf değil de printk?

Kernel'de standart C kütüphanesi (glibc) yoktur – modülün bağlı olduğu tek şey kernel'in kendisidir. `printk` kernelin kendi `printf` benzeri fonksiyonudur (kayan nokta – floating point – desteği **yoktur**, çünkü kernel modunda FP register durumunu kaydetme/geri yükleme maliyeti gereksiz bulunmuştur). `printk`'in `printf`'ten en büyük farkı: bir **öncelik seviyesi** (loglevel) alabilmesidir – bunu Bölüm 4'te detaylı göreceğiz.

Uygulama ile Modül Arasındaki Fark: Olay Güdümlü Model

Bir C uygulaması `main()`'den başlar, sırayla çalışır, biter. Bir kernel modülünün ise `main()`'i yoktur. `init` fonksiyonu "ben buradayım, şunları yapabilirim" der ve kaynakları/yeteneklerini **kaydedip hemen döner** – modülün asıl işi, kayıt ettiği fonksiyonlar (senin `fops` tablon gibi) başka biri tarafından *daha sonra* çağrıldığında yapılır. `exit` fonksiyonu ise "artık burada değilim" der, kayıt olan her şeyi geri alır.

Kernel Stack'inin Küçüklüğü

Kitabın açıkça vurguladığı bir tuzak: kernel stack'i çok küçüktür – bazı mimarilerde tek bir 4096 byte'lık sayfa kadar, ve bu stack **tüm kernel-alanı çağrı zincirinde paylaşılır**. Bu yüzden kernel kodunda **büyük yerel (otomatik/stack) değişkenler asla tanımlanmamalıdır** – örneğin fonksiyon içinde `char buf[8192];` gibi bir tanım kernel'de ciddi bir hatadır (stack overflow riski); büyük veri için `kmalloc` (Bölüm 8) kullanılmalıdır.

current: Şu Anki Process

`current` – `<asm/current.h>` üzerinden erişilen, o an çalışan process'i temsil eden `struct task_struct` * tipinde bir global işaretçi (aslında CPU başına ayrı tutulan, mimariye özel bir mekanizma, düz

bir global değişken değil). Örnek:

```
printk(KERN_INFO "The process is \"%s\" (pid %i)\n",
        current->comm, current->pid);
```

`current->comm` – çalıştıran programın adı (15 karaktere kırpılmış); `current->pid` – process ID. Bunu Bölüm 6'da `sculluid` örneğinde (kullanıcı bazlı erişim kontrolü) tekrar göreceksin.

Kbuild: Kernel Modülü Neden Normal gcc ile Derlenmez

Modülün, derlendiği kernel ile **birebir aynı ABI'ye, aynı yapı hizalamalarına** sahip olması gerekir – bu yüzden kendi başına `gcc dosya.c` ile derlenemez, kernel'in kendi build sistemi (*Kbuild*) kullanılır.

En basit `obj-m` kullanımı:

```
obj-m := hello.o
```

Birden fazla kaynak dosyadan tek modül üretmek istersen:

```
obj-m := module.o
module-objs := file1.o file2.o
```

Gerçek çağrı, kernel ağacının `makefile`'ını kullanarak yapılır:

```
make -C ~/kernel-2.6 M='pwd' modules
```

`-C` kernel kaynak dizinine geçer (üst-düzey `makefile`'ı bulur); `M=` o `makefile`'a "işini bitirince modül dizinine geri dön ve `obj-m` listesini kullanarak `modules` hedefini derle" der.

Kitabın önerdiği, kendini iki kez okuyan (bir kere doğrudan, bir kere Kbuild tarafından `KERNELRELEASE` tanımlıyken) idiyomatik `makefile` kalıbı:

```
ifneq ($(KERNELRELEASE),)
    obj-m := hello.o
else
    KERNELDIR ?= /lib/modules/$(shell uname -r)/build
    PWD := $(shell pwd)
default:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) modules
endif
```

insmod, modprobe, rmmod, lsmod – Gerçekte Ne Yaparlar

- **insmod** – modül kodunu+verisini kernel belleğine yükler; bir *linker* gibi davranır, modüldeki çözümlenmemiş sembolleri kernel'in sembol tablosuna karşı bağlar. Disk üzerindeki dosyayı değiştirmez, bellekteki bir kopya üzerinde çalışır. İçeride `sys_init_module` sistem çağrısını tetikler: kernel `vmalloc` ile bellek ayırır (Bölüm 8), modül metnini oraya kopyalar, kernel sembol tablosuna karşı referansları çözer, ve modülün `init` fonksiyonunu çağırır.
- **modprobe** – `insmod` gibi yükler, ama ek olarak modülün ihtiyaç duyduğu **başka modülleri de otomatik bulup yükler** (bağımlılık çözümü). Salt `insmod` kullanırsan ve eksik sembol varsa, sistem log dosyasında "unresolved symbols" hatası alırsın.
- **rmmod** – modülü kaldırır; modül hâlâ kullanımda görünüyorsa (örn. açık bir dosya tanıtıcısı ona referans veriyorsa) başarısız olur. Kitap açıkça uyarıyor: "zorla kaldırma" seçeneği olsa da, o noktaya gelmişsen muhtemelen işler zaten yeterince bozulmuştur – **yeniden başlatmak daha güvenli bir yoldur**.

- **lsmod** – yüklü modülleri listeler; `/proc/modules` dosyasını okuyarak çalışır (aynı bilgi `/sys/module` altında da mevcuttur).

Modül Parametreleri

Bir sürücünün sisteme özel bilgiye ihtiyacı olabilir (I/O adresi, DMA kanalı, bir tampon boyutu...) – bunları derleme zamanında sabitlemek yerine **yükleme zamanında** (`insmod/modprobe` ile) ayarlanabilir hale getirmek için `module_param` makrosu kullanılır:

```
#include <linux/moduleparam.h>

static char *whom = "world";
static int howmany = 1;
module_param(howmany, int, S_IRUGO);
module_param(whom, charp, S_IRUGO);
```

```
insmod hellop howmany=10 whom="Mom"
```

- **Desteklenen tipler:** `bool`, `invbool` (tersine çevrilmiş boolean), `charp` (char pointer – string), `int`, `long`, `short`, ve işaretsiz (u-önekli) karşılıkları.
- **Dizi parametreleri:** `module_param_array(name, type, num, perm)` – `num`, kaç değer girildiğini kaydeden bir `int` işaretçisi.
- **perm (izin) argümanı** – `<linux/stat.h>` içindeki tanımları kullanır, parametrenin `/sys/module/<isim>/parameters/` altındaki dosyasının izinlerini belirler:
 - 0 → `sysfs`'te hiç görünmez.
 - `S_IRUGO` → herkes okuyabilir, değiştirilemez.
 - `S_IRUGO|S_IWUSR` → root `sysfs` üzerinden değiştirebilir.

Kernel Sembol Tablosu ve Modül Yığılması (Stacking)

Bir modül başka bir modülün sembollerini kullanabilir – buna **modül yığılması** denir. Gerçek örnekler: `msdos` dosya sistemi `fat` modülünün sembollerini kullanır; paralel port alt sistemi `lp` → `parport` → `parport_pc` şeklinde katmanlanır. Bir sembolü dışa açmak için:

```
EXPORT_SYMBOL(isim);
EXPORT_SYMBOL_GPL(isim);
```

İkincisi sembolü sadece GPL lisanslı modüllere açar. Çoğu modülün hiç sembol dışa açmasına gerek yoktur – sadece başka modüllerin gerçekten faydalanacağı durumlarda kullanılır.

Hata Yönetimi: goto Kalıbı ve Sökme Sırası

Kayıt (registration) işlemleri başarısız olabilir (bellek yetersizliği gibi). Linux, hangi modülün neyi kaydettiğine dair **merkezi bir defter tutmaz** – yani bir kayıt başarısız olduğunda, o ana kadar başarıyla yapılmış olan *önceki* kayıtları geri almak (unregister) tamamen senin sorumluluğundadır. Kitap, iç içe `if`'lerden kaçınmak için normalde sevilmeyen `goto`'yu burada **bilinçli olarak önerir**:

```
int __init my_init_function(void)
{
    int err;
    err = register_this(ptr1, "skull");
    if (err) goto fail_this;
```

```

err = register_that(ptr2, "skull");
if (err) goto fail_that;
err = register_those(ptr3, "skull");
if (err) goto fail_those;
return 0; /* basari */

fail_those: unregister_that(ptr2, "skull");
fail_that:  unregister_this(ptr1, "skull");
fail_this:  return err;
}

```

Cleanup fonksiyonu, kayıtları **ters sırada** söker (A, B, C kaydedildiyse; C, B, A sökülür) – bu, kernel programlamada genel bir prensiptir ve senin `ahmet_exit`'inde de (`device_destroy` → `class_destroy` → `cdev_del` → `unregister_chrdev_region`) aynen uygulanmış durumda.

Bir modül bir yeteneği kaydettiği **anda**, kernelin başka bir parçası onu hemen kullanmaya başlayabilir – `init` fonksiyonun hâlâ çalışıyor olması bunu engellemez. **Kural: bir yeteneği, onu desteklemek için gereken tüm iç hazırlık tamamlanmadan kaydetme.**

Eğer `init`, bir yetenek zaten kaydedilip *kullanılmaya başlandıktan sonra* başarısız olursa, bu senaryoyu düşün: belki de hiç başarısız olmamak (`init`'i bitirmek) daha güvenlidir.

Kullanıcı Alanında mı, Kernel'de mi Yazmalı? (Doing It in User Space)

Kitap, çekingen kernel programcılarına şu soruyu sorduruyor: her zaman kernel modülü yazmak mı gerekir?

Kullanıcı alanı sürücüsünün avantajları (kitaptan, aynen):

1. Tam C kütüphanesi kullanılabilir.
2. Normal bir debugger (gdb) rahatça kullanılabilir.
3. Sürücü kilitlenirse sadece o process öldürülür, sistem genelde etkilenmez.
4. Kullanıcı belleği *swap edilebilir* – kernel belleği edilemez; nadiren kullanılan büyük bir sürücü sürekli RAM işgal etmez.
5. Kapalı kaynaklı sürücüler için lisans belirsizliği ortadan kalkar.

Dezavantajları:

1. Kesmeler (interrupt) user space'te mevcut değildir.
2. Doğrudan bellek erişimi sadece `/dev/mem`'i `mmap` ederek mümkün, ve bunu sadece yetkili kullanıcı yapabilir.
3. I/O port erişimi kısıtlı ve/veya yavaş.
4. Yanıt süresi daha yavaştır (context switch gerekir); sürücü disk'e swap edilmişse çok daha kötüleşir.
5. **En önemli cihaz sınıfları user space'te hiç yapılamaz:** ağ arayüzleri ve blok cihazlar bunlara dahildir.

Kitabın tavsiyesi: yeni/alışılmadık bir donanımı ilk öğrenirken user space'te denemek, tüm sistemi riske atmadan deney yapmayı sağlar – donanım anlaşıldıktan sonra "yazılımı bir kernel modülüne kapsüllemek sıkıntısız bir işlem olmalı."

Bölüm 2 – Hızlı Referans (kitabın kendi özeti)

- `insmod`, `modprobe`, `rmmod` – modül yükleme/kaldırma araçları.
- `module_init(f)`, `module_exit(f)` – init/exit fonksiyonlarını bildiren makrolar.
- `__init`, `__initdata`, `__exit`, `__exitdata` – yalnızca init/exit zamanında kullanılan fonksiyon/veri işaretleri; bellekten atılabilirler (özel bir ELF bölümüne yerleştirilerek).
- `struct task_struct *current;` – şu anki process.
- `obj-m` – Kbuild’e hangi modüllerin derleneceğini söyleyen makefile sembolü.
- `/sys/module`, `/proc/modules` – yüklü modüller hakkında bilgi.
- `EXPORT_SYMBOL(sym)`, `EXPORT_SYMBOL_GPL(sym)` – bir sembolü kernel’e (veya sadece GPL modüllere) açar.
- `module_param(degisken, tip, izin)` – yükleme-zamanı ayarlanabilir bir parametre tanımlar.
- `int printk(const char *fmt, ...);` – kernelin `printf` karşılığı.

Bölüm 2 Karakter Sürücüler

Bu bölüm kitabın en yoğun kod içeren bölümlerinden biridir çünkü artık gerçek bir sürücü (`scull`) baştan sona inşa edilir. Senin `ahmet.c`'n bu bölümün kavramlarının basitleştirilmiş bir uygulamasıdır – burada `scull`'un tam mimarisini görüp kendi kodunla karşılaştıracaksın.

`scull`'un Tasarımı: Neden Birden Fazla “Kişilik”?

`scull` = *Simple Character Utility for Loading Localities*. Gerçek donanıma değil, kernel belleğinden ayrılmış bir alana çalışır – bu yüzden taşınabilir, herkes derleyip çalıştırabilir. Kitap tek bir mekanizma üzerine **altı farklı “kişilik”** inşa eder, her biri gerçek dünyadaki bir erişim politikasını örnekler:

- **scull0–scull3**: dört cihaz, her biri hem *global* (aynı dosyayı açan tüm süreçler aynı veriyi paylaşır – senin `ahmet`'inin de yaptığı gibi) hem *kalıcı* (veri kapat/aç arasında kaybolmaz, sadece açık truncate ile silinir).
- **sculpipe0–3**: FIFO/boru gibi davranan dört cihaz – Bölüm 6'da bloklayan I/O örneği için kullanılır.
- **scullsingle**: `scull0` gibi ama **aynı anda sadece bir process** açabilir.
- **scullpriv**: her sanal konsol/terminale **özel** bir bellek alanı.
- **sculluid**: aynı anda sadece **bir kullanıcı** (UID) açabilir; ikinci kullanıcı hata alır.
- **scullwuid**: `sculluid` gibi ama hata vermek yerine **bekler** (bloklar).

Kitabın kendi ifadesiyle: bu erişim-kontrollü varyantlar "mekanizma ile politikayı karıştırıyormuş gibi görünebilir, ama gerçek hayattaki bazı cihazlar tam olarak bu tür yönetim gerektirir." Bu bölüm sadece `scull0–3`'ün iç yapısını kapsar; `sculpipe` ve erişim-kontrollü varyantlar Bölüm 6'da.

Major/Minor Numaralarının İç Temsili

`dev_t` tipi (`<linux/types.h>`), major ve minor'u tek bir sayıda taşır. 2.6.0'dan itibaren: 32 bit, **12 bit major, 20 bit minor**. Bu düzene asla doğrudan (bit kaydırma ile) güvenme – her zaman şu makroları kullan (`<linux/kdev_t.h>`):

```
MAJOR(dev_t dev);          /* major'u çıkar */
MINOR(dev_t dev);          /* minor'u çıkar */
MKDEV(int major, int minor); /* ikisinden dev_t oluşturun */
```

Kitap açıkça uyarıyor: `dev_t`'nin iç yapısı gelecekte tekrar değişebilir – taşınabilir kod bu makrolarla yazılmalı, ham bit işlemleriyle değil.

Cihaz Numarası Ayırma: `register_chrdev_region` vs `alloc_chrdev_region`

```
int register_chrdev_region(dev_t first, unsigned int count, char *name);
int alloc_chrdev_region(dev_t *dev, unsigned int firstminor,
                        unsigned int count, char *name);
void unregister_chrdev_region(dev_t first, unsigned int count);
```

- **register_chrdev_region**: major numarasını **sen zaten biliyorsan** kullanılır (`first` parametresiyle).
- **alloc_chrdev_region**: `dev` bir **çıktı** parametresidir – kernel dinamik olarak bir major seçer ve sonucu oraya yazar. Kitap açıkça şöyle diyor: **"yeni sürücülerin neredeyse kesinlikle**

`alloc_chrdev_region` kullanması gerekir" – rastgele boş bir major seçmek "sadece sürücünün tek kullanıcısı sen olduğun sürece işe yarar," sürücü yaygınlaştıkça çakışma riski artar.

Üç Temel Veri Yapısı: `file_operations`, `file`, `inode`

struct file_operations

Cihaz numaralarını sürücü koduna bağlayan tablo. Her açık dosyanın `f_op` alanı kendi `file_operations`'ına işaret eder. `__user` etiketi (örn. `char __user *buf`) bir işaretçinin **user-space adresi** olduğunu, kernel kodunun onu doğrudan dereference edemeyeceğini belgeler (statik analiz araçları için, derleyici için etkisi yoktur).

En çok kullanılan alanlar (NULL bırakılırsa ne olduğuyla birlikte):

- `owner` – işlem değil, `THIS_MODULE` olmalı; kullanımdayken modülün kaldırılmasını engeller.
- `read/write` – NULL ise `syscall -EINVAL` döner.
- `ioctl` – NULL ise öntanımlı olmayan her `ioctl` `-ENOTTY` döner.
- `open` – NULL ise açma her zaman başarılı olur ama sürücü haberdar edilmez.
- `release` – NULL olabilir; **her `close()` çağrısında değil**, sadece `file` yapısı gerçekten yok edildiğinde çağrılır (aşağıda detaylı).
- `llseek` – NULL ise kernel `f_pos`'u kendi varsayılan mantığıyla değiştirir (Bölüm 6'da detaylı).
- `mmap`, `poll`, `fsync`, `fsync` – ileri düzey; sırasıyla Bölüm 15 ve Bölüm 6'da.

`scull`'un gerçek tanımı, C99 adlandırılmış alan (designated initializer) söz dizimiyle:

```
struct file_operations scull_fops = {
    .owner =    THIS_MODULE,
    .llseek =   scull_llseek,
    .read =     scull_read,
    .write =    scull_write,
    .ioctl =    scull_ioctl,
    .open =     scull_open,
    .release =  scull_release,
};
```

Bu söz dizimi tercih edilir çünkü: (1) `struct`'ın alan sırası değişse bile kod kırılmaz, (2) belirtilmeyen alanlar otomatik NULL olur, (3) sık erişilen işaretçiler aynı cache satırına yerleştirilebilir.

struct file: f_pos, f_flags, private_data

`struct file` **açık bir dosyayı** temsil eder – C kütüphanesindeki `FILE*` ile **karıştırılmamalı**, ikisi birbirinden tamamen bağımsızdır. Sürücü yazarları için önemli alanlar:

- `f_pos` (`loff_t`, 64-bit) – güncel okuma/yazma konumu. Sürücü bunu **okuyabilir** ama normalde **doğrudan yazmamalı** – konum güncellemesi `read/write`'a geçirilen `*offp` işaretçisi üzerinden yapılmalı (istisna: `llseek`'in tüm işi zaten konum değiştirmektir).
- `f_flags` – `O_RDONLY`, `O_NONBLOCK`, `O_SYNC` gibi bayraklar. **Okuma/yazma izni kontrolü için `f_mode` kullanılmalı, `f_flags` değil.**
- `f_op` – kernel tarafından `open` sırasında atanır. **Kernel bu değeri başka hiçbir yerde ön-belleğe almaz** – yani bir sürücü kendi `open`'ı içinde `filp->f_op`'u farklı bir `file_operations`

tablosuna değiştirebilir, ve bu değişiklik o dosya tanıtıcısındaki sonraki tüm çağrılarda geçerli olur ("method overriding" – kernelin nesne-yönelimli hilesi). `/dev/null` ve `/dev/zero` (ikisi de major 1) aynı major altında farklı davranışlar sunmak için bunu kullanır.

- **private_data** – `open` syscall'ı tarafından `NULL`'a ayarlanır, sürücü serbestçe kullanabilir. **Bu senin `ahmet.c`'nin şu an kullanmadığı ama `scull`'un aktif kullandığı alan** – `scull`, `open` içinde kendi `scull_dev` yapısına işaretçiyi buraya koyar, böylece `read/write` her seferinde hangi cihaz örneğiyle uğraştığını bilir.

cdev: Sürücüyü Kernel'e Tanıtmak

```
void cdev_init(struct cdev *cdev, struct file_operations *fops);
int cdev_add(struct cdev *dev, dev_t num, unsigned int count);
void cdev_del(struct cdev *dev);
```

İki kullanım yolu: (1) `cdev_alloc()` ile bağımsız tahsis, ya da (2) **`scull`'un yaptığı gibi `cdev`'i kendi cihaz struct'ının içine gömmek**:

```
struct scull_dev {
    struct scull_qset *data; /* ilk quantum set'e isaretci */
    int quantum; /* guncel quantum boyutu */
    int qset; /* guncel dizi boyutu */
    unsigned long size; /* burada saklanan veri miktarı */
    unsigned int access_key; /* sculluid/scullpriv için */
    struct semaphore sem; /* karsilikli dislama semaforu */
    struct cdev cdev; /* Char device yapisi */
};
```

Kritik sıralama kuralı: `cdev_add()` döndüğü anda cihaz "canlıdır" – kernel operasyonlarını hemen çağırabilir. Bu yüzden `cdev_add`, sürücü tamamen hazır olmadan **asla** çağırılmamalı.

container_of: inode'dan kendi struct'ına geri dönmek

`scull`'un `open`'ı, `inode->i_cdev` üzerinden embed edilmiş `cdev`'e erişebilir – ama asıl istenen *onu* içeren `scull_dev`'dir. Çözüm, `<linux/kernel.h>`'deki genel amaçlı bir makro:

```
container_of(pointer, container_type, container_field);
```

```
int scull_open(struct inode *inode, struct file *filp)
{
    struct scull_dev *dev;
    dev = container_of(inode->i_cdev, struct scull_dev, cdev);
    filp->private_data = dev; /* diger metodlar için */

    if ( (filp->f_flags & O_ACCMODE) == O_WRONLY ) {
        scull_trim(dev); /* hatalari yoksay */
    }
    return 0;
}
```

`O_ACCMODE` maskesi: `O_WRONLY/O_RDONLY/O_RDWR` bayrakları temiz bir bit maskesi oluşturmadığı için, sadece erişim-modu bitlerini izole etmek amacıyla kullanılır. `scull`'un `open`'ının tek gerçek işi budur: dosya *sadece yazmak için* açılmışsa cihazı sıfıra kısaltmak (normal dosyaların `O_APPEND` olmadan yazma-açılışında truncate edilmesiyle aynı mantık).

scull'un Bellek Yönetimi: Quantum ve Qset

Bu bölüm zorunlu bir sürücü tekniği değil, **bir tasarım kararının** gösterimidir: scull cihaz boyutunu bilerek sınırlamaz (felsefi olarak keyfi sınır kötüdür; pratikte de düşük bellekli test için tüm RAM'i tüketebilmek faydalıdır – `cp /dev/zero /dev/scull0`).

Veri yapısı: her scull cihazı bir **bağlı listedir**, listenin her düğümü bir `scull_qset` yapısına işaret eder:

```
struct scull_qset {
    void **data;           /* pointer dizisi */
    struct scull_qset *next; /* bir sonraki dugum */
};
```

Terminoloji: bir işaretçinin gösterdiği bellek alanına **quantum** denir; işaretçi dizisinin kendisine (ya da uzunluğuna) **quantum set (qset)** denir. Öntanımlı olarak dizi 1000 işaretçi, her quantum 4000 byte'tır – yani bir düğüm 4 milyon byte'a kadar veri adresleyebilir.

`scull_trim()` tüm cihazın belleğini serbest bırakır (hem `scull_open`'ın `O_WRONLY truncate`'inde, hem modül kaldırılırken çağrılır):

```
int scull_trim(struct scull_dev *dev)
{
    struct scull_qset *next, *dptr;
    int qset = dev->qset;
    int i;
    for (dptr = dev->data; dptr; dptr = next) {
        if (dptr->data) {
            for (i = 0; i < qset; i++)
                kfree(dptr->data[i]);
            kfree(dptr->data);
            dptr->data = NULL;
        }
        next = dptr->next;
        kfree(dptr);
    }
    dev->size = 0;
    dev->quantum = scull_quantum;
    dev->qset = scull_qset;
    dev->data = NULL;
    return 0;
}
```

Bu fonksiyon listeyi baştan sona gezer; her düğümde önce her bir quantum'u, sonra dizinin kendisini, sonra düğümün kendisini serbest bırakır (sırayla – `next`'i düğümü silmeden önce kaydederek).

read ve write: Tam Algoritmik Detay

İmzalar:

```
ssize_t read(struct file *filp, char __user *buff,
             size_t count, loff_t *offp);
ssize_t write(struct file *filp, const char __user *buff,
             size_t count, loff_t *offp);
```

Neden buff'a doğrudan dokunulamaz? (3 gerekçe, kitaptan)

1. Mimariye ve kernel ayarlarına bağlı olarak, user-space işaretçisi kernel modunda çalışırken **hiç geçerli olmayabilir**.

2. Kullanıcı belleği **swap edilebilir** – işaretçiye doğrudan dokunmak beklenmedik bir page fault tetikleyebilir; kernel kodu bunu doğrudan yaşayamaz (bir "oops" ile process ölür).
3. **Güvenlik:** işaretçi kullanıcı tarafından verilir, hatalı ya da kötü niyetli olabilir. Körlemesine dereference etmek "bir user-space programının sistemin herhangi bir yerindeki belleğe erişmesine/üzerine yazmasına açık bir kapı" olur.

Bu yüzden `<asm/uaccess.h>`'deki özel, mimariye-güvenli fonksiyonlar zorunludur – senin zaten kullandığın `copy_to_user/copy_from_user`.

ssize_t dönüş değeri sözleşmesi (hem read hem write için)

- **Negatif** = hata (`<linux/errno.h>`'deki kodlardan biri).
- **count'a eşit** = istenen tüm veri aktarıldı (ideal durum).
- **Pozitif ama count'tan az** = kısmi aktarım; çağıran (örn. `fread`) kalanı almak için tekrar çağırmalı.
- **Sıfır:** read için = EOF (dosya sonu). write için = hiçbir şey yazılmadı ama bu **hata değildir** (bloklayan yazmanın tam anlamı Bölüm 6'da).

scull'un tam read kodu (semafor çağrıları Bölüm 5'e kadar "görmezden gel" diyerek tanıtılıyor):

```

ssize_t scull_read(struct file *filp, char __user *buf, size_t count,
                  loff_t *f_pos)
{
    struct scull_dev *dev = filp->private_data;
    struct scull_qset *dptr;
    int quantum = dev->quantum, qset = dev->qset;
    int itemsize = quantum * qset;
    int item, s_pos, q_pos, rest;
    ssize_t retval = 0;

    if (down_interruptible(&dev->sem))
        return -ERESTARTSYS;
    if (*f_pos >= dev->size)
        goto out;
    if (*f_pos + count > dev->size)
        count = dev->size - *f_pos;

    item = (long)*f_pos / itemsize;
    rest = (long)*f_pos % itemsize;
    s_pos = rest / quantum; q_pos = rest % quantum;

    dptr = scull_follow(dev, item);
    if (dptr == NULL || !dptr->data || !dptr->data[s_pos])
        goto out; /* bosluklari doldurma */

    if (count > quantum - q_pos)
        count = quantum - q_pos;

    if (copy_to_user(buf, dptr->data[s_pos] + q_pos, count)) {
        retval = -EFAULT;
        goto out;
    }
    *f_pos += count;
    retval = count;

out:
    up(&dev->sem);
    return retval;
}

```

```
}

```

Algoritma adım adım: önce `count`, `*f_pos + count` cihaz boyutunu aşmasın diye kırılır (EOF'un ötesine okuma yok). Sonra `item/s_pos/q_pos` – tam sayı bölme/mod ile `*f_pos`'un hangi liste düğümüne, hangi qset dizisi konumuna, hangi quantum-içi byte ofsetine denk geldiğini hesaplar. `scull_follow` listeyi gezip doğru düğümü bulur (yoksa NULL döner – "boşlukları doldurma" kuralı: eksik veri hata değil, EOF gibi davranılır). `count` bu kez *tek bir quantum* aşmayacak şekilde tekrar kırılır – **scull'un read'i bir çağrıda sadece tek bir quantum işler**, daha fazlası isteniyorsa çağırın tekrar çağırır (stdio bunu fark etmez bile).

`write` mantığı büyük ölçüde simetriktir, ama önemli bir farkla: `read` "boşluk" bulursa EOF gibi davranıp durur, ama `write` eksik olan `data` dizisini ve quantum'u **talep üzerine (on-demand) kmalloca ile ayırır** – cihaz büyüyebilir. Son olarak `dev->size`, yeni `*f_pos` eskisini aşıyorsa güncellenir.

scull ile Oynamak: strace Önerisi

Kitap, `cp`, `dd`, yönlendirme ile test etmeyi, ve özellikle **strace** kullanmayı önerir – örneğin `strace ls /dev > /dev/scull0` çalıştırırsan, `write` çağrısının 4096 byte istediğini ama scull'un quantum sınırı yüzünden 4088 byte kabul edip kalanı için tekrar çağrıldığını görürsün. Bu teknik Bölüm 4'te derinlemesine işlenir.

Bölüm 3 – Hızlı Referans

- `dev_t`, `MAJOR()`, `MINOR()`, `MKDEV()` – `<linux/types.h>`.
- `register_chrdev_region()`, `alloc_chrdev_region()`, `unregister_chrdev_region()` – `<linux/fs.h>`.
- `struct file_operations`, `struct file`, `struct inode` – üç temel yapı.
- `cdev_alloc()`, `cdev_init()`, `cdev_add()`, `cdev_del()` – `<linux/cdev.h>`.
- `container_of(ptr, tip, alan)` – `<linux/kernel.h>`.
- `copy_from_user()`, `copy_to_user()` – `<asm/uaccess.h>`.

Bölüm 3 Hata Ayıklama Teknikleri

Kernel’de hata ayıklamak user space’ten yapısal olarak farklıdır: kod tek bir process’e bağlı değildir, hatalı kod tüm sistemi çökertebilir, ve çökme "kanıtı" genelde kaybolur. Bu bölüm, sırasıyla *yazdırarak*, *sorgulayarak*, *izleyerek* ve son çare olarak *interaktif debugger ile* hata ayıklamayı ele alır.

printk: Log Seviyeleri

printk’in printf’ten temel farkı: format string’in başına bir **öncelik (loglevel)** dizgesi eklenebilir. Sekiz seviye vardır (<linux/kernel.h>), **azalan önem sırasıyla**:

KERN_EMERG	sistem kullanılamaz duruma gelmeden hemen önce
KERN_ALERT	derhal müdahale gerektiren durum
KERN_CRIT	kritik durumlar, ciddi donanım/yazılım hatası
KERN_ERR	hata durumu (sürücüler donanım sorunları için kullanır)
KERN_WARNING	ciddi olmayan ama sorunlu durumlar
KERN_NOTICE	normal ama dikkate değer durumlar
KERN_INFO	bilgilendirme (örn. açılıшта bulunan donanım)
KERN_DEBUG	hata ayıklama mesajları

Loglevel belirtilmezse öntanımlı KERN_WARNING kullanılır. Mesaj, kendi öncelik sayısı console_loglevel değişkeninden **küçükse** konsola da basılır; klogd/syslogd çalışıyorsa mesaj bağımsız olarak /var/log/messages’da yazılır. console_loglevel değiştirme yöntemleri: klogd -c <seviye>, ya da doğrudan /proc/sys/kernel/printk dosyasına yazmak (echo 8 > /proc/sys/kernel/printk → debug dahil her şeyi konsola bas).

printk nasıl saklanır: dairesel tampon (circular buffer)

printk, verisini sabit boyutlu bir **daireesel tampona** yazar (__LOG_BUF_LEN, 4KB–1MB arası, derleme zamanı seçimi). Tampon dolarsa **en eski veri üzerine yazılır** – bilinçli bir tercih: bu sayede bir loglama süreci (process) olmadan da sistem çalışabilir, bellek de israf edilmez. dmesg bu tamponu **tüketmeden** (okuyup silmeden) dump eder; klogd’un kullandığı /proc/kmsg okuması ise tamponu tüketir (FIFO gibi davranır – ikinci bir okuyucu varsa çakışma olur).

printk_ratelimit: log taşmasını önlemek

Dikkatsiz printk kullanımı binlerce mesaj üretip konsolu boğabilir; **yavaş konsol cihazlarında** (örn. seri port) sistemi **yavaşlatabilir hatta kilitleyebilir** (kesme kullanılamayan bir seri konsolu sürmek için). **Kural: üretim kodu normal çalışmada asla yazdırmamalı, sadece istisnai durumlarda.** Tekrarlayan hata durumları için kernel bir hız sınırlayıcı sağlar:

```
if (printk_ratelimit())
    printk(KERN_NOTICE "The printer is still on fire\n");
```

Ayarlanabilir eşikler: /proc/sys/kernel/printk_ratelimit (saniye), /proc/sys/kernel/printk_ratelimit_bu (eşiğe kadar kabul edilen mesaj sayısı).

Koşullu Debug Mesajları: PDEBUG Kalıbı

Kitabın scull kaynağında kullandığı, derleme-zamanı açılıp kapanabilen debug makrosu:

```
#undef PDEBUG
#ifdef SCULL_DEBUG
#   ifdef __KERNEL__
#       define PDEBUG(fmt, args...) printk( KERN_DEBUG "scull: " fmt, ## args)
#   else
#       define PDEBUG(fmt, args...) fprintf(stderr, fmt, ## args)
#   endif
#endif
```

```
# endif
#else
# define PDEBUG(fmt, args...) /* debug yok: hiçbir sey */
#endif
```

Bu, gcc'nin değişken-argümanlı makro genişletmesini kullanır. Kernel alanında `printk`, kullanıcı alanında (aynı kodu test programı olarak derlersen) `fprintf(stderr,...)` olur – **aynı debug makrosu iki dünyada da çalışır**. Makefile'da `-DSCULL_DEBUG` bayrağıyla açılır/kapanır – yani debug mesajlarını değiştirmek **yeniden derleme** gerektirir (buna karşın çalışma-zamanı if koşuluyla yapılsaydı her zaman ufak bir performans maliyeti olurdu, ama yeniden derleme gerekmezdi – kitap bu ödünleşimi (trade-off) açıkça belirtir).

/proc Dosya Sistemi ile Sorgulama

Sürekli `printk` basmak yerine, durumu **talep üzerine** (`ps`, `netstat` gibi) sorgulamak daha az müdahaleci bir yöntemdir. `/proc`, her dosyası bir kernel fonksiyonuna bağlı, yazılım tarafından anlık üretilen sanal bir dosya sistemidir.

Basit salt-okunur bir `/proc` girdisi için (2.6.10-dönemi API):

```
int (*read_proc)(char *page, char **start, off_t offset, int count,
                 int *eof, void *data);

struct proc_dir_entry *create_proc_read_entry(const char *name,
                                              mode_t mode, struct proc_dir_entry *base,
                                              read_proc_t *read_proc, void *data);
```

Kernel, sürücünün veri yazması için `PAGE_SIZE` boyutunda bir tampon (`page`) ayırır; `count/offset` normal `read` ile aynı anlama gelir; `eof` sürücünün "veri bitti" diye işaretlediği bir `int` işaretçisidir. Modül kaldırılırken `remove_proc_entry()` ile girdi **mutlaka** silinmelidir.

Büyük çıktılar için modern/tercih edilen yaklaşım `seq_file` arayüzüdür (`<linux/seq_file.h>`) – `start/next/stop/show` adlı dört yineleyici (iterator) fonksiyonu uygulanır, çıktı `seq_printf` ile üretilir (ham `printk` ile değil). Kitap: "birden fazla küçük miktarda çıktı üreten her dosya için `seq_file` kullanılmalı."

ioctl ile Hata Ayıklama

`/proc`'a alternatif: sürücüye özel bir debug `ioctl` komutu ekleyip iç veri yapılarını user space'e kopyalamak. Avantajı: `/proc` dosyaları dizin listelemesinde **herkese görünür** (meraklı gözlerin dikkatini çeker), oysa belgelenmemiş bir `ioctl` komutu fark edilmesi zor ama gerektiğinde hâlâ oradadır. Dezavantajı: kullanmak için ayrı bir user-space programı yazman gerekir.

strace: Sistem Çağrılarını Dışarıdan İzlemek

`strace`, bir user-space programının yaptığı **tüm sistem çağrılarını** – isim, argümanlar, dönüş değeri – sembolik biçimde gösterir. Kernel'in kendisinden bilgi aldığı için, izlenen programın `-g` ile derlenmiş olması gerekmez.

Oops Mesajlarını Okumak

Çoğu kernel hatası NULL işaretçi dereference'ı ya da geçersiz bir işaretçi değeri şeklinde ortaya çıkar → bir **oops** mesajı. **Oops** ≠ **panic**: Linux dayanıklıdır, bir oops genelde sadece o anki process'i öldürür, sistem çalışmaya devam eder (panic ise tüm sistemi durdurur, çok daha nadirdir).

Örnek (kitaptan, `faulty` modülünün `*(int*)0 = 0;` satırından):

```
Unable to handle kernel NULL pointer dereference at virtual address 00000000
Oops: 0002 [#1]
EIP: 0060:[<d083a064>] Not tainted
EIP is at faulty_write+0x4/0x10 [faulty]
...
Call Trace:
[<c0150558>] vfs_write+0xb8/0x130
[<c0150682>] sys_write+0x42/0x70
```

Okuma stratejisi (kitabın kendi sırası):

1. **Önce EIP is at ... satırına bak** – hatanın tam olarak nerede olduğunu (fonksiyon adı + modül adı + fonksiyon içindeki offset) sembolik olarak gösterir. Bu örnekte: `faulty_write` fonksiyonu, `faulty` modülü, fonksiyonun başından 4 (hex) byte içeride, fonksiyon toplam 0x10 byte uzunlukta.
2. Yeterli değilse, **Call Trace** (çağrı yığını) bloğuna bak – oraya nasıl gelindiğini gösterir.
3. **"Tainted" bayrağına dikkat et** – GPL-olmayan/güvenilmez bir modül yüklenmişse "Tainted" görünür, bu da hata raporunun ciddiye alınma olasılığını etkiler.

Sistem Kilitlenmeleri: Magic SysRq

Bazı hatalar oops bile vermeden sistemi **tamamen kilitler**. Kod sonsuz bir döngüye girerse, kernel zamanlamayı (scheduling) durdurabilir – sistem Ctrl-Alt-Del dahil hiçbir şeye yanıt vermez. Önleyici teknik: uzun döngülerin içine stratejik noktalarda `schedule()` çağrılarını eklemek (**ama asla bir spinlock tutarken `schedule()` çağırma**).

Magic SysRq tuşu (Alt+SysRq+komut), çoğu kilitlenmede vazgeçilmez bir araçtır – `CONFIG_MAGIC_SYSRQ` ile etkinleştirilmelidir (çoğu dağıtımda güvenlik nedeniyle **varsayılan olarak kapalıdır**). Faydalı komutlar:

- **r** – klavyeyi raw moddan çıkarır (çökmüş bir X sunucusunun bıraktığı garip klavye durumunu düzeltir).
- **s** – tüm diskleri acil senkronize eder (sync).
- **u** – tüm diskleri salt-okunur yeniden bağlar (umount); genelde **s**'den hemen sonra, dosya sistemi kontrolü süresini kısaltmak için.
- **b** – anında yeniden başlatır (önce **s** ve **u** çalıştırılmalı!).
- **p** – işlemci register bilgisini yazdırır.
- **t** – güncel görev (task) listesini yazdırır.

Debugger'lar: gdb, kdb, kgdb, UML

- **gdb** + `/proc/kcore`: `gdb /usr/src/linux/vmlinux /proc/kcore - /proc/kcore`, tüm kernel adres alanını temsil eden, okundukça *üretilen* özel bir dosyadır. **Sınırlama**: gdb veri okur ama **önbelleğe alır** – `p jiffies` gibi değişen bir değeri tekrar sorgularsan **eski (baya)** veri görebilirsin; `core-file /proc/kcore` komutunu tekrar vererek önbelleği temizlemen gerekir. Ayrıca gdb bu modda **breakpoint koyamaz, veri değiştiremez, tek adım ilerleyemez** – sadece okuma amaçlıdır. `CONFIG_DEBUG_INFO` gerektirir.

- **kdb**: resmi kernel'e dahil değildir (Linus Torvalds interaktif debugger'lara felsefi olarak karşıdır – "semptomu yamalamayı, kök nedeni bulmayı değil teşvik ediyor" endişesiyle), ayrı bir yama olarak dağıtılır, sadece x86'da çalışır.
- **kgdb**: gerçek breakpoint/tek-adım desteği için iki ayrı makine kullanır – test edilen kernel bir "kurban" makinede, gdb'nin kendisi **stabil bir masaüstünde** çalışır, aralarında genelde bir **seri kablo** bağlantısı vardır.
- **User-Mode Linux (UML)**: kernelin kendisini normal bir user-space process'i olarak çalıştıran ayrı bir port. Avantajı: hatalı kernel gerçek sistemi bozamaz. **Kritik kısıt**: UML kernelinin gerçek donanıma **erişimi yoktur** – donanımla konuşan sürücü kodunu debug etmek için kullanılamaz.

Bölüm 4 – Ön Plana Çıkan Uyarılar

- Debug config seçenekleri (`CONFIG_DEBUG_*`) ekstra çıktı üretir ve performansı düşürür – bu yüzden üretim/dağıtım kernellerinde bilinçli olarak kapalıdır.
- `CONFIG_INPUT_EVBUG` **her tuş vuruşunu, şifreler dahil** loglar – açık bir güvenlik uyarısı.
- `/proc` girdisi kaldırmayı unutmak, modül kaldırılırken beklenmedik çökmelere yol açar.
- `schedule()` çağrıları reentrant çağrılara kapı açar – asla bir spinlock tutarken çağrılmamalı.

Bölüm 4 Eşzamanlılık ve Race Condition'lar

scull'daki Somut Race Condition

Kitap, scull'un `write` kodundan gerçek bir parça üzerinden race condition'ı gösterir:

```
if (!dptr->data[s_pos]) {
    dptr->data[s_pos] = kmalloc(quantum, GFP_KERNEL);
    if (!dptr->data[s_pos])
        goto out;
}
```

Senaryo: İki process, A ve B, **aynı scull cihazının aynı ofsetine** aynı anda yazıyor.

1. Her ikisi de `if (!dptr->data[s_pos])` testine **aynı anda** ulaşır.
2. İşaretçi NULL olduğu için **ikisi de** bellek ayırmaya karar verir ve sonucu aynı yere atar.
3. İki atama da aynı konuma yazdığı için, sadece **sonuncusu** "kazanır."
4. scull, **ilk (kaybeden) process'in ayırdığı belleği tamamen unuttur** – bu bellek asla serbest bırakılmaz → **bellek sızıntısı**.

Kitabın tanımı: "**Race condition, paylaşılan veriye kontrolsüz erişimin bir sonucudur. Yanlış erişim örüntüsü gerçekleştiğinde, beklenmedik bir şey olur.**" Ve önemli bir uyarı: "**Programcılar race condition'ları son derece düşük olasılıklı olaylar olarak görmezden gelmeye meyillidir. Ama bilgi işlem dünyasında, milyonda bir olay her birkaç saniyede bir gerçekleşebilir, ve sonuçları ciddi olabilir.**"

Genel Kural: Paylaşılan Kaynak = Yönetim Zorunluluğu

Kitabın "sert kural"ı (tam ifadeyle): "**bir donanım ya da yazılım kaynağı tek bir çalışma iş parçacığının (thread of execution) ötesinde paylaşıldığında, ve bir iş parçacığının o kaynağın tutarsız bir görünümüyle karşılaşma olasılığı varsa, o kaynağa erişimi açıkça yönetmek zorundasın.**"

Eşzamanlılığın kaynakları (kitabın listesi):

1. Birden fazla user-space process'in sürücü koduna "şaşırtıcı kombinasyonlarda" erişmesi.
2. SMP sistemlerde farklı işlemcilerin sürücü kodunu **gerçekten aynı anda** çalıştırması.
3. **Kernel preemption** – 2.6 kernel kodu önceliklenebilir (preemptible); sürücü kodun herhangi bir anda işlemciyi kaybedebilir, ve yerine geçen process de aynı sürücü kodunda olabilir.
4. Kesmeler (interrupt) – asenkron olarak sürücü kodunun eşzamanlı çalışmasına neden olur.
5. Ertelenmiş çalıştırma mekanizmaları (workqueue, tasklet, timer).
6. Takılıp-çıkartılabilir (hot-pluggable) cihazlar – cihaz tam onunla uğraşırken ortadan kaybolabilir.

Semaphore ve Mutex

Kritik bölge (critical section): "aynı anda sadece bir iş parçacığı tarafından çalıştırılabilecek kod."

```
DECLARE_MUTEX(name);          /* 1'e ilklendirilmiş */
DECLARE_MUTEX_LOCKED(name);   /* 0'a ilklendirilmiş (kilitli baslar) */
void init_MUTEX(struct semaphore *sem);
```

```
void init_MUTEX_LOCKED(struct semaphore *sem);

void down(struct semaphore *sem);           /* kesilemez uyku */
int down_interruptible(struct semaphore *sem); /* sinyal ile kesilebilir */
int down_trylock(struct semaphore *sem);     /* hic uyumaz */
void up(struct semaphore *sem);
```

scull'da semafor kullanımı

```
struct scull_dev {
    ...
    struct semaphore sem; /* karsilikli dislama semaforu */
    struct cdev cdev;
};

for (i = 0; i < scull_nr_devs; i++) {
    scull_devices[i].quantum = scull_quantum;
    scull_devices[i].qset = scull_qset;
    init_MUTEX(&scull_devices[i].sem);
    scull_setup_cdev(&scull_devices[i], i);
}
```

Neden global tek bir semafor değil, cihaz-başına semafor? Kitap: "çeşitli scull cihazları hiçbir kaynağı ortak kullanmıyor... bir process farklı bir scull cihazıyla çalışırken diğerini beklemeye zorlamak için hiçbir gerekçe yok" – paralelliği artırır.

Reader/Writer Semaphore (rwsem)

Çoğu görev salt-okunur/yazma olarak ayrışır; `struct rw_semaphore` birden fazla eşzamanlı okuyucuya (yazar yokken) izin verir:

```
void down_read(struct rw_semaphore *sem);
int down_read_trylock(struct rw_semaphore *sem); /* NOT: basari=sifirdan
FARKLI, tersi degil! */
void up_read(struct rw_semaphore *sem);
void down_write(struct rw_semaphore *sem);
void up_write(struct rw_semaphore *sem);
```

rwsem'lerde yazarlar önceliklidir – bir yazar beklemeye başladığında yeni okuyucular kabul edilmez, bu da **okuyucu açlığına** (reader starvation) yol açabilir. Bu yüzden rwsem'ler sadece yazma erişiminin nadir ve kısa olduğu durumlarda tercih edilmeli.

Completion: "İş Bitti" Bildirimi

Yanlış yaklaşım (kitabın açıkça uyardığı): bir işi başlatıp kilitli bir semaforda `down()` ile beklemek, işi yapan taraf `up()` çağırınca devam etmek. Bu yanlış çünkü: (1) semaforlar "müsait" (uncontended) hızlı yol için optimize edilmiştir, oysa burada çağırana neredeyse **her zaman** bekleyecektir – tam tersi senaryo; (2) semafor bir **stack (otomatik) değişkeni** olarak tanımlanmışsa, bekleyen taraf fonksiyonundan dönüp semaforu yok ederken, sinyal veren taraf hâlâ ona dokunuyor olabilir – ciddi bir race condition.

Bu yüzden **completion** arayüzü eklenmiştir:

```

DECLARE_COMPLETION(my_completion);
/* veya dinamik: */
struct completion my_completion;
init_completion(&my_completion);

void wait_for_completion(struct completion *c); /* kesilemez bekleme! */
void complete(struct completion *c);          /* sadece TEK bekleyeni uyandırır */
void complete_all(struct completion *c);      /* TUM bekleyenleri uyandırır */

```

Spinlock: Uyuyamayan Kod İçin Kilit

Semaforun aksine spinlock **uyku gerektirmez/desteklemez** – kesme işleyicileri gibi uyuyamayan bağlamlarda kullanılır. Kilidi alamayan kod, müsait olana kadar sıkı bir döngüde bekler ("spin eder").

```

spinlock_t my_lock = SPIN_LOCK_UNLOCKED; /* derleme-zamani */
spin_lock_init(&my_lock);                /* çalışma-zamani */

void spin_lock(spinlock_t *lock);
void spin_lock_irqsave(spinlock_t *lock, unsigned long flags);
void spin_lock_irq(spinlock_t *lock);
void spin_lock_bh(spinlock_t *lock);      /* sadece softirq/tasklet'i kapatır */
/*

void spin_unlock(spinlock_t *lock);
void spin_unlock_irqrestore(spinlock_t *lock, unsigned long flags);
void spin_unlock_irq(spinlock_t *lock);
void spin_unlock_bh(spinlock_t *lock);
*/

```

Kilit tutma süresi kuralı: spinlock'lar **mümkün olan en kısa süre** tutulmalı – uzun tutulan bir kilit, diğer işlemcilerin bekleme süresini ve zamanlama gecikmesini (scheduling latency) doğrudan kötüleştirir.

Kilitleme Tuzakları

- **Self-deadlock:** bir fonksiyon bir kilidi alıp, sonra **aynı kilidi** tekrar almaya çalışsan başka bir fonksiyonu çağırırsa, kod kilitlenir – ne semaphore ne spinlock, bir kilit sahibinin kilidi ikinci kez almasına izin vermez, basitçe asılı kalır.
- **Sıralama (ordering) deadlock'u:** iki kilit (Lock1, Lock2) varsa ve bir thread önce Lock1 sonra Lock2 alırken, başka bir thread önce Lock2 sonra Lock1 almaya çalışırsa **ikisi de kilitlenir**. **Çözüm:** birden fazla kilit alınacaksa, **her zaman aynı sırayla** alınmalı. Genel kural: yerel (kendi) kilidini, kernelin daha merkezi bir kilidinden **önce** al; semaforları spinlock'lardan **önce** al (bir spinlock tutarken **down** çağırmak – yani uyuyabilecek bir şey çağırmak – "ciddi bir hatadır").
- **Büyük Kernel Kilidi (BKL):** Linux'un ilk SMP-uyumlu kerneli (2.0) tek bir spinlock'a sahipti – tüm kerneli tek bir dev kritik bölgeye çeviriyordu. 2.6'da hâlâ (çok azaltılmış biçimde) var, ama kitap açıkça uyarıyor: **"yeni kodda onu kullanmayı aklından bile geçirme."**
- **İnce mi kaba mı taneli kilitleme:** kitabın tavsiyesi sürücü yazarları için nettir: **"gerçek bir çekişme (contention) sorunu olacağına dair somut bir sebep olmadıkça, görece kaba taneli kilitlemeyle başla. Erken optimizasyon dürtüsüne direnç göster; gerçek performans kısıtlamaları genelde beklenmedik yerlerde ortaya çıkar."**

Kilitlemeye Alternatifler

- **Dairesel tampon (lock-free)**: tek yazar/tek okuyucu senaryosunda, yazar veriyi **önce** tamponun içine yazıp **sonra** yazma-indeksini güncellerse, okuyucu her zaman tutarlı bir görünüm görür – **hiç kilide gerek kalmadan**. 2.6.10'dan itibaren kernel'de hazır bir uygulaması var: `<linux/kfifo.h>`.
- **atomic_t**: tek bir tamsayı üzerinde atomik işlemler (`<asm/atomic.h>`). **24 bitten fazlasına güvenme** (bazı mimarilerin sınırlaması).

```
atomic_t v = ATOMIC_INIT(0);
atomic_set(&v, i); atomic_read(&v);
atomic_inc(&v); atomic_dec(&v);
int atomic_dec_and_test(atomic_t *v); /* sonuc 0 ise true doner */
```

- **Bit işlemleri**: `set_bit`, `clear_bit`, `test_and_set_bit` vb. (`<asm/bitops.h>`) atomik bit erişimi sağlar. Kitap bit-tabanlı elle kilitlemeyi **caydırır**: "yeni kodda spinlock kullanmak çok daha iyidir; spinlock'lar iyi test edilmiştir, kesme/preemption gibi konuları halleder, ve kodunu okuyan başkalarının ne yaptığını anlamak için uğraşmasına gerek kalmaz."
- **RCU (Read-Copy-Update)**: okumanın sık, yazmanın nadir olduğu durumlar için ileri düzey, kilitsiz bir teknik – veri sadece **işaretçi üzerinden** erişilmeli. Yazar değişikliği önce bir **kopya** üzerinde yapar, sonra işaretçiye tek bir atomik adımda yeni kopyaya yönlendirir; eski kopya, tüm işlemcilerin en az bir kez zamanlanmasıyla sonra (artık kimse referans tutmuyor garantisıyla) `call_rcu()` ile serbest bırakılır. Gerçek kullanım örneği: ağ yönlendirme tabloları.

Bölüm 5 – Ön Plana Çıkan Uyarılar

1. Kilitleri, korunan nesneyi sistemin geri kalanına açmadan **önce** ilklendir.
2. Her `down/spin_lock` için, **hata yollarını dahil** tam olarak bir eşleşen `up/spin_unlock`.
3. Spinlock tutarken asla uyuma – `copy_from_user` ve `kmalloc(GFP_KERNEL)` gibi gizli uyku noktalarına dikkat.
4. Aynı-CPU'da kesme kaynaklı kendine-kilitlenmeyi önlemek için gerektiğinde `_irqsave/_irq` varyantlarını kullan.
5. Birden fazla kilidi hep **aynı sırada** al; semaforları spinlock'lardan önce al.
6. BKL'yi (`lock_kernel`) yeni kodda asla kullanma.
7. Erken optimizasyona direnç göster – kaba taneli kilitle başla, gerçek çekişme kanıtlanınca inceltmeyi düşün.

Bölüm 5 Gelişmiş Karakter Sürücü İşlemleri

ioctl: Genel Bakış

Kullanıcı tarafı imza: `int ioctl(int fd, unsigned long cmd, ...)`; Buradaki ... gerçek bir değişken-argüman listesi **değildir** (sistem çağrılarında değişken argüman sayısı alamaz) – sadece derleyicinin üçüncü argüman üzerinde tip kontrolünü **kapatmak** için oradadır. Gerçekte tek bir `unsigned long` argüman iletilir; kullanıcı bir işaretçi geçirirse kernel onu `unsigned long` olarak alır.

Çoğu ioctl uygulaması tek bir büyük `switch(cmd)` bloğudur.

Komut Numarası Kodlamaşı: Neden Bu Kadar Karmaşık?

Temel gerekçe (kitaptan): ioctl komut numaraları **sistem genelinde benzersiz** olmalıdır – aksi halde bir program yanlışlıkla *yanlış cihaza* doğru numaralı ama farklı anlamdaki bir komut gönderebilir (örneğin bir FIFO'ya, bir seri portun baud rate komutunu göndermek gibi). Her numara benzersizse, yanlış eşleşme `-EINVAL` ile başarısız olur – yanlışlıkla **istenmeyen bir işlemi çalıştırmak yerine**.

Modern kodlama şeması dört bit alanından oluşur (`<asm/ioctl.h>`):

- **type** (magic number) – sürücü başına bir sayı, 8 bit.
- **number** – sıra numarası, 8 bit, kendi başına bir anlamı yok.
- **direction** – **uygulamanın bakış açısından** veri yönü: `_IOC_NONE`, `_IOC_READ` (uygulama cihazdan okuyor), `_IOC_WRITE` (uygulama cihaza yazıyor), ya da ikisinin OR'u.
- **size** – veri boyutu; kernel bunu **zorunlu kılmaz** ama iyi pratik olarak kullanıcı hatalarını yakalamaya yarar.

Kodlama makroları:

```
_IO(type, nr)           /* argumansız komut */
_IOR(type, nr, datatype) /* driver'dan OKUMA */
_IOW(type, nr, datatype) /* driver'a YAZMA */
_IOWR(type, nr, datatype) /* iki yonlu */
```

scull'un tanımladığı komutlar:

```
#define SCULL_IOC_MAGIC  'k'
#define SCULL_IOCRESET   _IO(SCULL_IOC_MAGIC, 0)
#define SCULL_IOCSQUANTUM _IOW(SCULL_IOC_MAGIC, 1, int) /* Set: pointer ile */
#define SCULL_I OCTQUANTUM _IO(SCULL_IOC_MAGIC, 3)      /* Tell: deger ile */
#define SCULL_IOC GQUANTUM _IOR(SCULL_IOC_MAGIC, 5, int) /* Get: pointer ile */
#define SCULL_IOC QQUANTUM _IO(SCULL_IOC_MAGIC, 7)      /* Query: return
degeriyle */
#define SCULL_IOC_MAXNR 14
```

Dönüş değeri kuralı: bilinmeyen cmd için POSIX'e göre `-ENOTTY` döndürülmeli ("Inappropriate ioctl for device") – tarihsel olarak sadece tty'lerin ioctl'i olduğu dönemden kalma (`-EINVAL` de pratikte sık görülür ama standart olan `-ENOTTY`'dir).

ioctl Argümanının Güvenli Kullanımı

arg bir işaretçiyse, dereference etmeden önce doğrulanmalıdır:

```
int access_ok(int type, const void *addr, unsigned long size);
/* type: VERIFY_READ veya VERIFY_WRITE (WRITE, READ'in ustkumesidir) */
/* DONUS: 1 = basari, 0 = basarisizlik -- co u kernel fonksiyonunun TERSI! */
```

scull'un komut doğrulaması, hem sayısal geçerliliği hem yön bitlerini kontrol eder:

```
if (_IOC_TYPE(cmd) != SCULL_IOC_MAGIC) return -ENOTTY;
if (_IOC_NR(cmd) > SCULL_IOC_MAXNR) return -ENOTTY;

if (_IOC_DIR(cmd) & _IOC_READ)
    err = !access_ok(VERIFY_WRITE, (void __user *)arg, _IOC_SIZE(cmd));
else if (_IOC_DIR(cmd) & _IOC_WRITE)
    err = !access_ok(VERIFY_READ, (void __user *)arg, _IOC_SIZE(cmd));
if (err) return -EFAULT;
```

Tek bir değer aktarmak için, access_ok zaten geçtiyse daha hızlı makrolar kullanılabilir:

```
__put_user(datum, ptr); /* access_ok'u ATLAR, onceden dogrulanmis olmalı */
__get_user(local, ptr);
put_user(datum, ptr); /* kendi icinde access_ok cagirir */
get_user(local, ptr);
```

Capabilities: root Yerine İnce Taneli Yetki

Geleneksel Unix modeli hep-ya-da-hiç'tir (sadece root). Linux bunun yerine **capabilities** sunar – ayrıcalıklı işlemler alt gruplara bölünür. Sürücü yazarları için önemli olanlar: CAP_SYS_ADMIN (genel yönetim, "daha spesifik bir capability yokluğunda" en sık kullanılan), CAP_SYS_MODULE, CAP_SYS_RAWIO, CAP_DAC_OVERRIDE.

```
int capable(int capability); /* surecin bu yetkiye sahip olup olmadigini
dondurur */

case SCULL_IOC_SQUANTUM:
    if (!capable(CAP_SYS_ADMIN))
        return -EPERM;
    retval = __get_user(scull_quantum, (int __user *)arg);
    break;
```

scull'un politikası: herkes quantum/qset boyutunu **sorgulayabilir** (Get/Query), ama sadece yetkili kullanıcılar **değiştirebilir** (Set/Tell) – kötü değerler performansı bozabilir.

Blocking I/O: Process'i Uyutmak

Üç temel kural (kitaptan, aynen)

1. **Atomik bağlamda asla uyuma.** Bir spinlock, seqlock, ya da RCU kilidi tutarken, ya da kesmeler kapalıyken uyuyamazsın. Bir semafor tutarken uyumak **yasal** ama çok kısa olmalı ve o semaforu tutmanın seni uyandıracak tarafı bloklamadığından emin olmalısın.
2. **Uyandıktan sonra sistem durumu hakkında hiçbir varsayım yapma.** Ne kadar uyuduğunu, o sırada neyin değiştiğini bilemezsin; başka bir process aynı olay için uyuyor olabilir ve senden önce kaynağı "kazanmış" olabilir. **Beklediğin koşulu her zaman yeniden kontrol etmelisin.**
3. **Process'in uyanacağından emin olmadan uyuma.** Bir yerde, birinin seni uyandırması garanti olmalı.

wait_event ailesi

```
DECLARE_WAIT_QUEUE_HEAD(name);          /* derleme-zamani */
wait_queue_head_t q; init_waitqueue_head(&q); /* calisma-zamani */

wait_event(queue, condition);
wait_event_interruptible(queue, condition); /* tercih edilen */
wait_event_timeout(queue, condition, zamanasimi);
wait_event_interruptible_timeout(queue, condition, zamanasimi);
```

```
void wake_up(wait_queue_head_t *queue);
void wake_up_interruptible(wait_queue_head_t *queue);
```

sleepy örneği: paylaşılan bayrak race'i

```
static DECLARE_WAIT_QUEUE_HEAD(wq);
static int flag = 0;

ssize_t sleepy_read (struct file *filp, char __user *buf, size_t count, loff_t *
pos)
{
    wait_event_interruptible(wq, flag != 0);
    flag = 0;
    return 0;
}
ssize_t sleepy_write (struct file *filp, const char __user *buf, size_t count,
loff_t *pos)
{
    flag = 1;
    wake_up_interruptible(&wq);
    return count;
}
```

O_NONBLOCK ve -EAGAIN

filp->f_flags içindeki O_NONBLOCK bayrağı ayarlıysa, veri/yer yokken read/write hemen -EAGAIN döndürmelidir – process'i asla bloklamaz. Sadece read, write, ve open bu bayraktan etkilenir.

scullpipe: Gerçek Bir Bloklayan Okuma Örnek

```
struct scull_pipe {
    wait_queue_head_t inq, outq; /* okuma ve yazma kuyruklari */
    char *buffer, *end;
    int buffersize;
    char *rp, *wp; /* nereden oku, nereye yaz */
    int nreaders, nwriters;
    struct semaphore sem;
    struct cdev cdev;
};
```

İki ayrı wait queue: inq (okuyucular veri bekler), outq (yazıcılar yer bekler) – kuralda geçen "farklı işlemler için farklı kuyruk" ilkesinin somut hali.

```
static ssize_t scull_p_read (struct file *filp, char __user *buf,
size_t count, loff_t *f_pos)
{
    struct scull_pipe *dev = filp->private_data;
    if (down_interruptible(&dev->sem))
```

```

    return -ERESTARTSYS;

    while (dev->rp == dev->wp) { /* okunacak bir sey yok */
        up(&dev->sem);           /* KILIDI BIRAK -- yoksa yazicilar bizi hic
                                uyandıramaz */
        if (filp->f_flags & O_NONBLOCK)
            return -EAGAIN;
        if (wait_event_interruptible(dev->inq, (dev->rp != dev->wp)))
            return -ERESTARTSYS;
        if (down_interruptible(&dev->sem)) /* uyandik: kilidi TEKRAR al */
            return -ERESTARTSYS;
        /* dongu basina donup kosulu YENIDEN kontrol et */
    }
    /* ...veriyi kopyala, rp'yi ilerlet... */
    up(&dev->sem);
    wake_up_interruptible(&dev->outq); /* yazicilari uyandır */
    return count;
}

```

Elle Uyuma Mekanizması (wait_event'in altında ne oluyor)

1. Bir wait_queue_t girdisi oluşturulup kuyruğa eklenir.
2. Process durumu değiştirilir: TASK_RUNNING (çalışabilir), TASK_INTERRUPTIBLE/TASK_UNINTERRUPTIBLE (uyuyor). **Sadece durumu değiştirmek process'i uyutmaz** – zamanlayıcıya nasıl davranacağını söyler, henüz CPU bırakılmamıştır.
3. **Kritik desen:** durum ayarlandıktan hemen sonra, ama schedule() çağrılmadan önce koşul tekrar kontrol edilir:

```

if (!condition)
    schedule();

```

Bu, olayın *ne zaman* gerçekleştiğine bakılmaksızın doğru çalışır: koşul durum ayarlanmadan **önce** doğru olduysa, bu kontrolde yakalanır (uyumadan atlanır); **sonra** gerçekleşirse, process zaten çalıştırılabilir yapılmıştır.

4. schedule() – zamanlayıcıyı çağırır, CPU'yu bırakır.

poll ve select

Bir process'in bloklamadan "okuyabilir miyim / yazabilir miyim" sorabilmesini sağlayan mekanizma. Tek bir sürücü metoduyla desteklenir:

```

unsigned int (*poll) (struct file *filp, poll_table *wait);

```

İki görevi vardır: (1) poll_wait()'i durumdaki değişikliği gösterebilecek her wait queue için çağır-mak, (2) hangi işlemlerin **bloklamadan** tamamlanabileceğini gösteren bir bit maskesi döndürmek (POLLIN|POLLRDNORM okunabilir, POLLOUT|POLLWRNORM yazılabilir, POLLHUP EOF, POLLERR hata).

```

static unsigned int scull_p_poll(struct file *filp, poll_table *wait)
{
    struct scull_pipe *dev = filp->private_data;
    unsigned int mask = 0;

    down(&dev->sem);
    poll_wait(filp, &dev->inq, wait);
    poll_wait(filp, &dev->outq, wait);
}

```

```

    if (dev->rp != dev->wp)
        mask |= POLLIN | POLLRDNORM;
    if (spacefree(dev))
        mask |= POLLOUT | POLLWRNORM;
    up(&dev->sem);
    return mask;
}

```

Asenkron Bildirim (fasync / SIGIO)

Kullanıcı tarafı iki adımlı bir fcntl anlaşmasıdır:

```

fcntl(fd, F_SETOWN, getpid()); /* 1. kimi bilgilendireceğini söyle
*/
fcntl(fd, F_SETFL, fcntl(fd, F_GETFL) | FASYNC); /* 2. bildirimi aç */

```

Ardından kernel, dosyada yeni veri geldiğinde sahibi SIGIO sinyali gönderir. Sürücü tarafında hazır yardımcı fonksiyonlar kullanılır:

```

int fasync_helper(int fd, struct file *filp, int mode, struct fasync_struct **
fa);
void kill_fasync(struct fasync_struct **fa, int sig, int band);

```

scull örneği:

```

static int scull_p_fasync(int fd, struct file *filp, int mode)
{
    struct scull_pipe *dev = filp->private_data;
    return fasync_helper(fd, filp, mode, &dev->async_queue);
}
/* write icinde, yeni veri geldiginde: */
if (dev->async_queue)
    kill_fasync(&dev->async_queue, SIGIO, POLL_IN);

```

Cihazı Aramaya Kapatmak: nonseekable_open

.llseek NULL bırakılırsa, kernelin varsayılan davranışı yine de aramaya (seek) izin verir – yani seri port gibi gerçek bir veri akışı sunan (veri alanı değil) bir cihaz için, aramayı devre dışı bırakmak istiyorsan .llseek'i boş bırakmak yetmez. Açıkça reddetmek gerekir:

```

int nonseekable_open(struct inode *inode, struct file *filp);
/* .open icinde cagır, ve fops'ta: */
.llseek = no_llseek,

```

scull, gerçek bir veri alanı olduğu için kendi .llseek'ini uygular – SEEK_END durumunda cihazın güncel boyutunu (dev->size) kullanarak yeni konumu hesaplar.

Cihaz Dosyası Üzerinde Erişim Kontrolü

Dosya izin bitleri kimin bir cihazı açabileceğini kısıtlar – ama bazen yetkili kullanıcılar arasında bile aynı anda sadece biri açık tutabilmelidir. Kitap dört farklı politikayı ayrı scull varyantları olarak gösterir:

- **scullsingle** – atomic_t sayaç ile ("brute-force," kitabın kendi tabiriyle "genelde kaçınılması gereken" bir yaklaşım – meşru eşzamanlı kullanım desenlerini (örn. bir okuma-durumu process'i + bir yazma-verisi process'i) engeller):

```
static atomic_t scull_s_available = ATOMIC_INIT(1);
if (! atomic_dec_and_test (&scull_s_available)) {
    atomic_inc(&scull_s_available);
    return -EBUSY;
}
```

- **sculluid** – sadece **tek bir kullanıcı** (birden fazla process’e izin vererek) açabilir; spinlock ile korunan sayaç + sahip uid:

```
spin_lock(&scull_u_lock);
if (scull_u_count &&
    (scull_u_owner != current->uid) &&
    (scull_u_owner != current->euid) &&
    !capable(CAP_DAC_OVERRIDE)) {
    spin_unlock(&scull_u_lock);
    return -EBUSY; /* -EPERM kullanıcıyı yanlış yöne yönlendirir */
}
```

- **scullwuid** – sculluid gibi ama -EBUSY yerine bir wait queue’da **bekler**; cron gibi arka plan görevleri için etkileşimli reddetme daha uygun olabilir. Kitap gerçek dünya örneği verir: Linux teyp sürücüsü, aynı fiziksel cihaz için **farklı davranışlar** sunan birden fazla cihaz düğümü (device node) kullanır – bloklama-vs-reddetme gibi uyumsuz politikaların her ikisine de ihtiyaç varsa, çözüm genelde tek bir cihazı iki politikayla zorlamak değil, **iki ayrı cihaz düğümü** sunmaktır.
- **scullpriv** – her açan process’e (kontrol eden terminaline göre anahtarlanan) **ayrı bir sanal kopya** sunar – sadece scull gibi saf yazılım cihazlarında mümkün. Anahtar olarak terminal yerine uid kullanılsaydı, her kullanıcıya özel bir sanal cihaz elde edilirdi – bu tamamen bir politika seçimidir.

Bölüm 6 – Ön Plana Çıkan Uyarılar

1. ioctl komut numaraları asla küçük ardışık tam sayılar olmamalı – type/number/direction/size kodlamasına uyulmalı.
2. access_ok’un VERIFY_READ/VERIFY_WRITE anlamı, ioctl’in kendi yön bitlerine göre **tersine** döner.
3. __put_user/__get_user sadece access_ok ile önceden doğrulanmış adreslerde kullanılmalı.
4. Check-then-sleep race’i: koşulu kontrol edip sonra uyumak iki **ayrı, senkronize olmayan** adımsa, arada gelen bir uyandırma sonsuza kadar kaybolabilir – wait_event* bunu içeride otomatik çözer, elle kod yazıyorsan sen çözmelisin.
5. Beklemeden önce, uyandırıcı tarafın ihtiyaç duyduğu kilidi **bırak**.
6. Sinyalle kesilme her zaman -ERESTARTSYS ile bildirilmeli.
7. .llseek’i boş bırakmak aramayı devre dışı **bırakmaz** – aramayı reddetmek için açıkça nonseekable_open + no_llseek gerekir.

Bölüm 6 Bellek Ayırma

Senin `ahmet.c`'in şu an `static char buffer[1024]` ile **derleme-zamanı** sabit bir alan kullanıyor. Bu bölüm, kernel'in **çalışma-zamanında** bellek istemenin kaç farklı yolu olduğunu ve hangisini ne zaman seçmen gerektiğini anlatır.

kmalloc: Kernel'in malloc'u, Ama Farklı Kurallarla

`kmalloc(size_t size, int flags)` çağrısı **fiziksel olarak bitişik** (contiguous) bellek döndürür – ve **önceki içeriğini temizlemez**; user space'e gönderilecek ya da cihaza yazılacaksa elle sıfırlanmalı (aksi halde önceki kernel verisi sızabilir).

En önemli iki bayrak, ne zaman hangisinin kullanılacağı konusunda net bir kural taşır:

- **GFP_KERNEL** – normal, process bağlamındaki (process context) çağrılar için. Bellek azsa **uyuyabilir** (process'i bekletir). Bu yüzden onu çağırın fonksiyon **reentrant olmalı ve atomik bağlamda çalışmamalıdır**.
- **GFP_ATOMIC** – process bağlamı **dışında** (kesme işleyicisi, tasklet, timer) zorunludur. **Asla uyumaz**; kernel bunun için ayrı bir acil-durum rezervi tutar, o da tükenirse ayırma **başarısız olur** (ama hiç uyumaz).

Boyut sınırı: `kmalloc`, önceden tanımlı sabit boyut sınıflarından (2'nin katları) bellek verir – istediğinden biraz **fazlasını** alabilirsin (en kötü ihtimalle 2 katı). Taşınabilir kod **128 KB**'tan fazlasını `kmalloc` ile isteyemez – sistem çalıştıkça bitişik büyük blok bulmak zorlaşır (parçalanma/fragmentation).

Slab Cache: Aynı Boyuttaki Nesneleri Sık Sık Ayırıp Bırakanlar İçin

Bir sürücü **aynı boyutta** nesneleri sürekli ayırıp serbest bırakıyorsa (USB ve SCSI sürücüler gibi), `kmalloc` yerine `kmem_cache_create()` ile kendi havuzunu (lookaside cache) oluşturabilir – `kmem_cache_alloc/kmem_cache_free` ile ayırıp bırakır, `kmem_cache_destroy` ile kapatır (sadece tüm nesneler geri döndüyse başarılı olur – başarısız dönüş, bir sızıntı olduğunun işaretidir). Kitabın `scullc` varyantı bunu gösterir: `kmalloc` yerine cache kullanmak hız değil, **daha öngörülebilir/-verimli bellek kullanımı** sağlar.

Sayfa Bazlı Ayırma ve vmalloc: Farklı İhtiyaçlar İçin Farklı Araçlar

`__get_free_pages()` ailesi, `kmalloc`'un aksine tam sayfa(lar) ister – büyük parçalar için `kmalloc`'tan daha verimlidir (`kmalloc`'un öngörülemeyen fazlalığı yoktur).

CPU-Başına Değişkenler: Kilitli Hız

2.6 kernel'in bir özelliği: `DEFINE_PER_CPU(tip, isim)` ile tanımlanan bir değişkenin **her işlemci kendi kopyasına** sahip olur – kitleme gerekmeden hızlı sayaç/istatistik tutmak için idealdir (ağ paket sayaçları gibi).

Büyük Tamponlar: Neden Kaçınmalı

Kitap açık bir tavsiye veriyor: büyük, bitişik bellek ayırmaları **zamanla arızaya eğilimlidir** (sistem çalıştıkça bellek parçalanır). Gerçekten büyük bir tampon gerekiyorsa, en iyi çözüm genelde tek bir dev blok istemek yerine **scatter/gather** işlemleridir (Bölüm 15'in konusu, bu belgenin kapsamı dışında).

Bölüm 7 Kesme (Interrupt) Yönetimi

Kesme Neden Var: Polling'e Alternatif

Donanım, "bir şey oldu" demek için CPU'yu sürekli yokladırarak (polling – boşa CPU harcar) yerine bir **kesme sinyali** gönderir; CPU o an ne yapıyorsa bırakıp senin kayıtlı fonksiyonunu çalıştırır. Kesme hatları (IRQ) sınırlı bir kaynaktır – kullanmadan önce talep edilmeli:

```
int request_irq(unsigned int irq, irq_handler_t handler,
               unsigned long flags, const char *dev_name, void *dev_id);
void free_irq(unsigned int irq, void *dev_id);
```

`dev_id`'nin iki görevi vardır: (1) `free_irq`'nin **hangi** kaydı sileceğini bilmesi, (2) **paylaşılan** bir IRQ hattında, kesme geldiğinde hangi cihaz örneğinin ilgilendiğini ayırt etmek. Bir handler'ın dönüş değeri: `IRQ_HANDLED` (cihazım gerçekten kesme üretti, ilgilendim) ya da `IRQ_NONE` (bu kesme benim cihazımdan gelmedi).

Handler Kuralları: Neden Bu Kadar Kısıtlı

Bir kesme işleyicisi, Bölüm 5'teki spinlock kurallarının en katı haliyle çalışır: **user space'e erişemez, uyuyamaz, `schedule()` çağıramaz**. Bu yüzden handler'ın işi minimal tutulmalı – donanımdan gelen veriyi bir tampona kaydetmek, gerekiyorsa donanıma "aldım" demek, ve ağır işi ertelemek.

Kesme ile Bloklayan I/O'nun Buluştuğu Yer

Bölüm 6'da öğrendiğin bloklayan `read`'in gerçek dünyadaki karşılığı tam olarak budur: veri yokken process `wait_event_interruptible` ile uyur; veri geldiğinde **kesme işleyicisi** minimum işi yapar ve `wake_up_interruptible()`'ı çağırarak bekleyen process'i uyandırır. Yani Bölüm 6'daki "kim uyandıracak?" sorusunun donanım dünyasındaki cevabı: **kesme işleyicisi**.

Paylaşılan Kesmeler

Birden fazla cihaz aynı IRQ hattını paylaşabilir (`IRQF_SHARED/SA_SHIRQ`). Kesme geldiğinde **o hatta kayıtlı tüm handler'lar çağrılır** – bu yüzden her handler önce **kendi cihazının** gerçekten kesme ürettiğini (bir durum register'ını okuyarak) kontrol etmeli, değilse hemen `IRQ_NONE` dönmelidir. `dev_id` bu yüzden paylaşımlı hatlarda **asla NULL olamaz** – kernel onu handler'ları birbirinden ayırt etmek için kullanır.

Bölüm 10 – Ön Plana Çıkan Uyarılar

- Modül kaldırılırken `free_irq()` çağrılmazsa, bir sonraki kesme artık geçersiz/kaldırılmış koda işaret eder – klasik bir use-after-free türü hata.
- Handler'da fazla iş yapmak, sadece o cihazı değil, sistemin genel tepki süresini bozar – ağır iş her zaman ertelenmeli.
- Paylaşımlı bir hatta `enable_irq/disable_irq` çağırarak **diğer cihazları** da etkiler – yapılmamalı.
- Handler ile process-bağlamındaki kod aynı veriyi paylaşıyorsa, process tarafı `spin_lock_irqsave` kullanılmalı (Bölüm 5) – aksi halde aynı CPU'da kendini kilitleyebilirsin.

Bölüm 8 The Linux Device Model (Aygıt Modeli)

kobject: Her Şeyin Altında Yatan Ortak İskelet

Kernel, cihaz modelindeki hemen her nesneyi (`struct cdev` dahil – Bölüm 3’te gördüğün!) bir `struct kobject` gömerek inşa eder. `kobject`’in tek işi üç şeydir: **referans sayımı** (kaç yerin bu nesneye ihtiyacı var), **sysfs’te bir dizin olarak görünme**, ve **hotplug olayı üretme**.

sysfs: Neden Her kobject Bir Dizindir

`kobject`’in sistemdeki konumu (`parent` işaretçisi üzerinden) doğrudan `/sys` altında bir **dizine** eşlenir. Bir `kobject`’in sunduğu her **attribute** (özellik), o dizinde bir **dosya** olarak görünür – `show/store` fonksiyonlarıyla okunur/yazılır. Kitabın vurguladığı tasarım kuralı: **"her dosya tek bir, insan tarafından okunabilir değer içermeli"** – büyük/karmaşık veri birden fazla dosyaya bölünmeli.

Hotplug Olayı: udev’in /dev Düğümünü Nasıl Oluşturduğu

Bir `kobject` sisteme eklendiğinde/çıkarıldığında kernel bir **hotplug olayı** (`uevent`) üretir – bu, user-space’teki `udev`’i tetikler. **Mekanizmanın tam olarak senin /dev/ahmet’inle ilgisi burada:** `udev`, bildirim aldığı anda `/sys/class/...` ağacında `dev` adlı bir dosya arar – bu dosya `major:minor` numarasını içerir. **Bir sürücünün `udev` için yapması gereken tek şey, kontrol ettiği cihazların `major/minor` numaralarını `sysfs` üzerinden dışa açmaktır** – `class_create` + `device_create` çağrılarının tam olarak bunu yapıyor: bir sınıf (`/sys/class/ahmet_class`) oluşturup, altına `dev` dosyasını içeren bir cihaz girdisi ekliyor.

Bus → Device → Driver: PCI/USB’de Gördüğünün Genelleşmiş Hali

Bir **bus** (`struct bus_type`), üzerindeki cihazları ve sürücülerini iki kset olarak tutar; bir `match()` fonksiyonu hangi sürücünün hangi cihaza uyduğuna karar verir. Eşleşme bulunduğunda sürücünün `probe()` fonksiyonu çağrılır – Bölüm 12/13’te PCI ve USB için gördüğün "cihaz tak, doğru sürücü otomatik eşleşsin" deseninin **soyut, genel çerçevesi** budur.

class: `class_create`’in geçmişi ve senin kodunla farkı

```
class_simple_create(owner, name) → class_create(owner, name) → class_create(name)
```

Senin kodunda `class_create("ahmet_class")` – **tek argümanlı** – kullanılıyor, yani sen kitaptan **iki nesil sonraki** bir API’deyken (`owner` parametresi tamamen kaldırılmış). İsimler değişse de **mekanizma birebir aynı**: sınıf oluştur → `/sys/class/<isim>` dizini oluşturun → cihaz ekle → `dev` dosyası ve hotplug olayı üretilsin → `udev /dev` düğümünü yaratsın.

Firmware Yükleme (Kısa Not)

Donanımın çalışabilmesi için önce bir firmware dosyası yüklenmesi gerekiyorsa, kitap firmware’i **kaynak koduna gömmeyi** açıkça yanlış bulur (lisans sorunu + güncelleme zorluğu). Doğru yol: `request_firmware()` ile user space’ten çalışma zamanında istemek – bu çağrı **uyuyabilir** (process bağlamı gerektirir), uyuyamayan bağlamlar için `request_firmware_nowait()` (callback tabanlı) vardır.

Bölüm 14 – Ön Plana Çıkan Uyarılar

- Bir `kobject` içeren `struct`'i asla doğrudan `kfree` etme – sadece `release` callback'i içinde, `kobject_put` tetiklediğinde.
- `sysfs` dosyaları "her dosyada tek değer" kuralına uymalı.
- `show/store` fonksiyonları `user space`'ten gelen veriyi asla güvenilir kabul etmemeli – geçersizse negatif hata dön.
- Kitaptaki `class_simple_*` isimlerini gördüğünde paniklemeyin – bugünkü `class_create/device_create`'i doğrudan atalarıdır, aynı mekanizmayı farklı isimle sunar.
- Firmware'i asla kaynak koduna gömme – hem lisans hem güncellenebilirlik sorunu yaratır.

Sonuç: Kitaptan ahmet.c'ye Dönen Bir Yol Haritası

Bu sekiz bölümü okuduktan sonra, kendi sürücünü aşamalı olarak geliştirmek için önerilen sıra – her adım tek bir bölümün tek bir kavramına karşılık geliyor, yani her adımı bitirdiğinde *hangi bölümü uyguladığını* tam olarak bileceksin:

1. **(Bölüm 5)** `ahmet_write/ahmet_read` içine bir `struct mutex` ekle (modern karşılığı – `DECLARE_MUTEX` değil). `ahmet_open`'da değil, modülün `ahmet_init`'inde, cihaz sisteme açılmadan **önce** ilklendir – Bölüm 5'teki sıralama kuralını birebir uygula.
2. **(Bölüm 3)** `ahmet_write`'i `*offset`'i kullanacak şekilde düzelt – `scull_write`'ın algoritmasını (kırpma + kopyalama + `*offset` güncelleme) birebir örnek al.
3. **(Bölüm 6)** Basit bir `ioctl` komutu ekle (örneğin `AHMET_IOCRESET` – tamponu sıfırlayan, argümansız bir `_IO` komutu). Bu, hem komut kodlamasını hem `.ioctl/.unlocked_ioctl` metodunu eklemeyi gerektirecek.
4. **(Bölüm 6)** `ahmet_read`'i, veri yokken `wait_event_interruptible` ile bekleyecek şekilde değiştir – `scull_p_read`'in "kilidi bırak, bekle, kilidi geri al, koşulu yeniden kontrol et" desenini uygula.
5. **(Bölüm 2 + 8)** `BUFFER_SIZE`'i bir `module_param`'a çevir, `kmalloc/kfree` ile dinamik ayır – `ahmet_init` process bağlamında çalıştığı için `GFP_KERNEL` doğru bayraktır.
6. **(Bölüm 14)** `class_create/device_create` çağrılarının kitaptaki `class_simple_create` ile aynı mekanizma olduğunu bilerek kodunu tekrar oku – `sysfs`'teki `/sys/class/ahmet_class/ahmet/dev` dosyasını (`cat /sys/class/ahmet_class/ahmet/dev`) inceleyip `udev`'in bu dosyadan nasıl `/dev/ahmet`'i ürettiğini gözlemle.
7. **(Bölüm 10, ileri seviye)** Staj kapsamında gerçek bir donanıma (GPIO, buton, sensör) bağlanırsan, `request_irq` ile bir kesme işleyicisi ekle; handler'da sadece minimum işi yap ve Bölüm 6'daki `wait_event_interruptible` ile bloklanan bir `read`'i `wake_up_interruptible` ile uyandır – Bölüm 6 ve Bölüm 10'un birleştiği tam nokta budur.
8. **(Bölüm 4)** Her adımdan sonra `dmesg` çıktısını ve gerekirse `strace`'i kullanarak davranışı doğrula – özellikle `ioctl` ve bloklayan `read` eklemelerinde, `strace` sana `syscall` seviyesinde tam olarak ne olduğunu gösterecek.